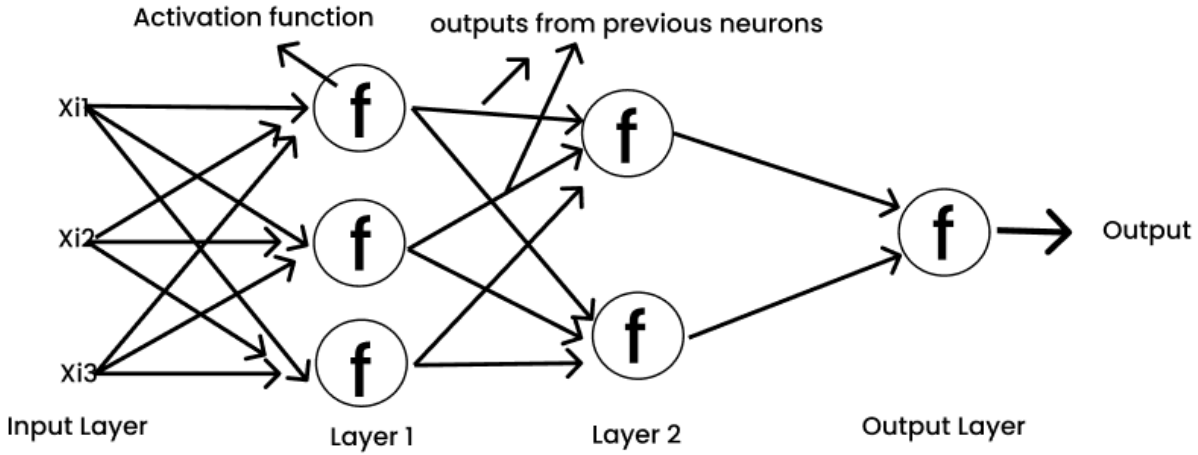
Perceptrón y Redes Neuronales Multicapa

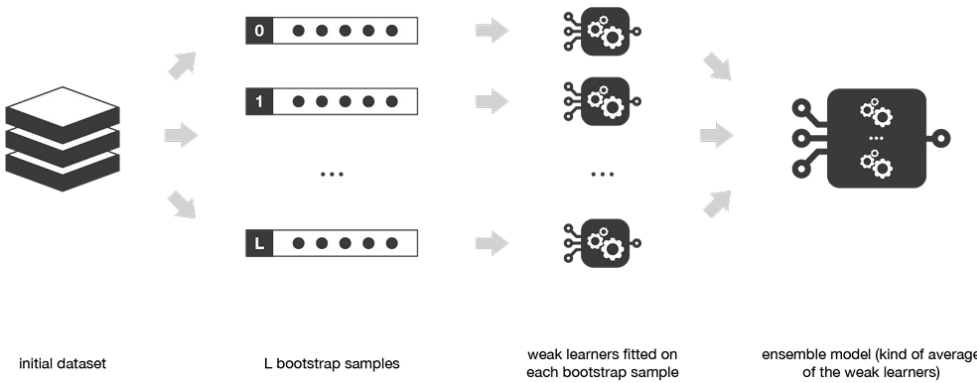
Joan Raygoza

De los ensambles a las redes neuronales

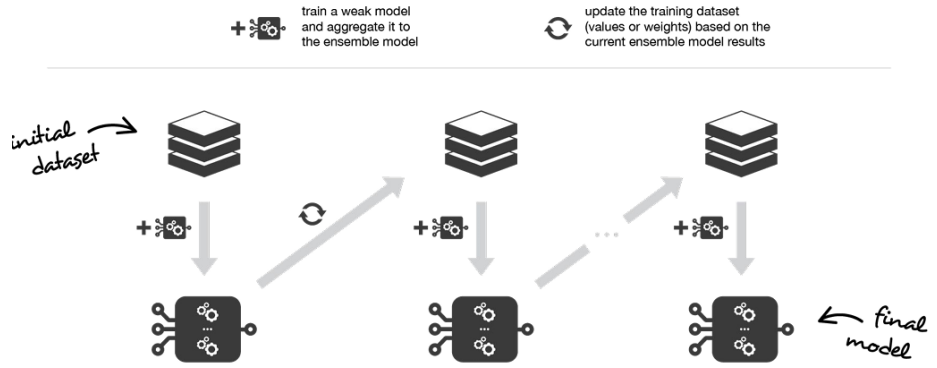
- Si ya sabemos combinar modelos para mejorar una predicción, ¿qué cambiaría si las piezas intermedias aprendieran juntas a partir del error final?



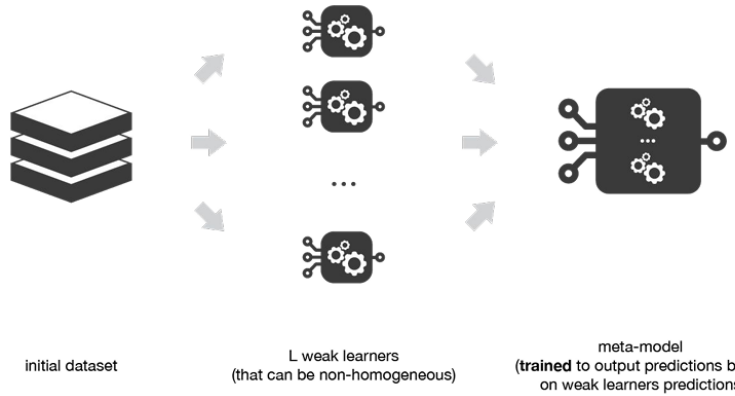
Multi layer perceptron



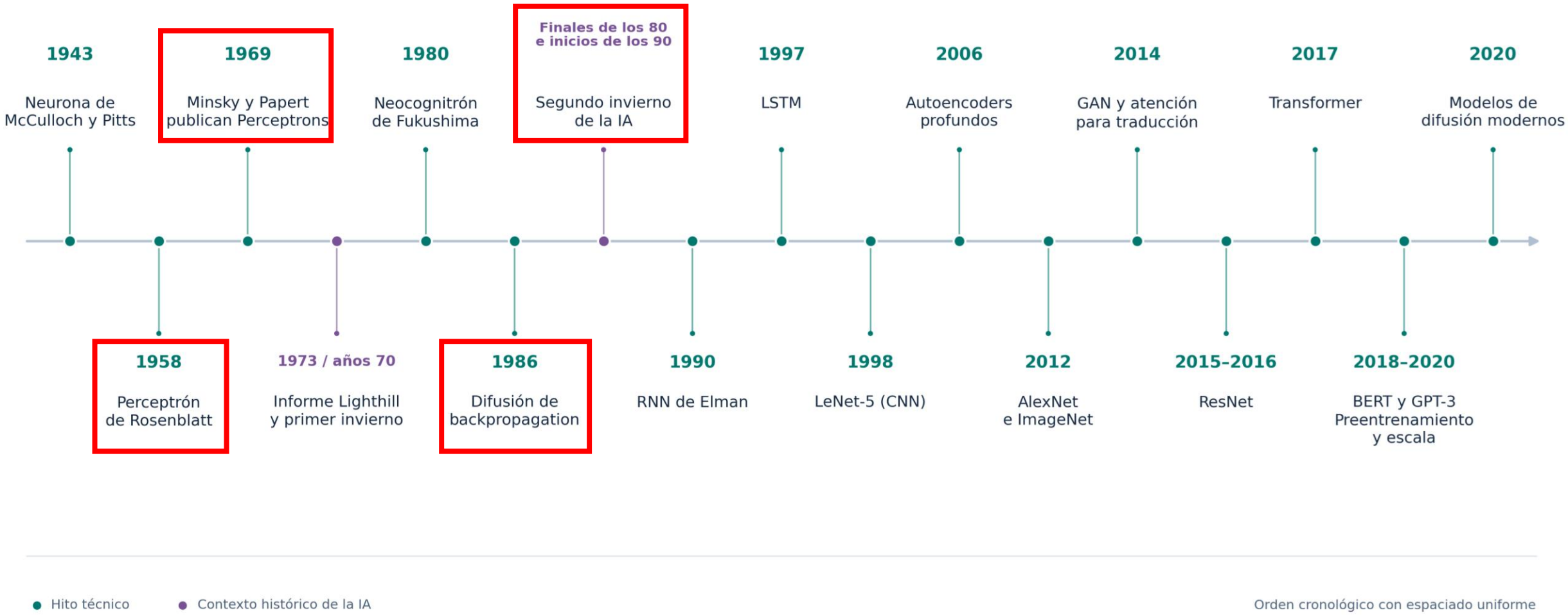
Bagging

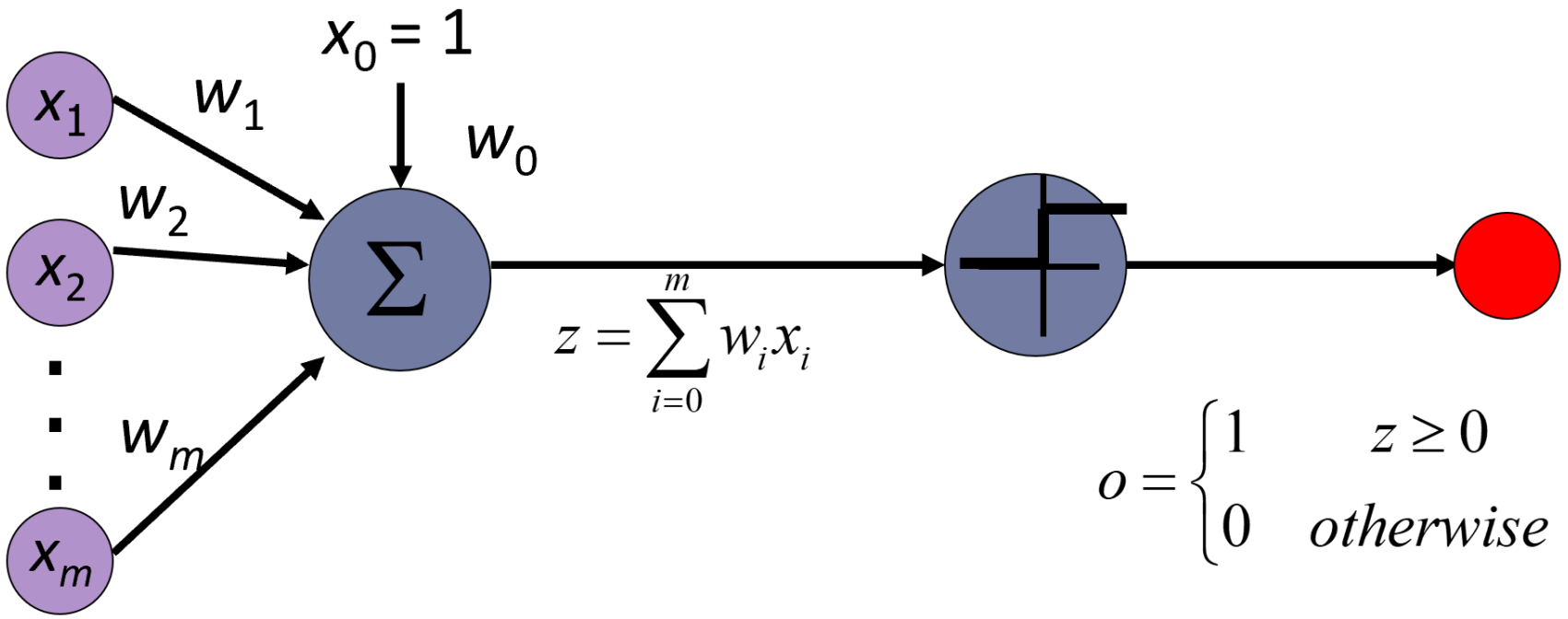


Boosting



Stacking


1943
Neurona de McCulloch y Pitts
1958
Perceptrón de Rosenblatt
1969
Minsky y Papert publican Perceptrons
1973 / años 70
Informe Lighthill y primer invierno
1980
Neocognitrón de Fukushima
1986
Difusión de backpropagation
Finales de los 80 e inicios de los 90
Segundo invierno de la IA
1990
RNN de Elman
1997
LSTM
1998
LeNet-5 (CNN)
2006
Autoencoders profundos
2012
AlexNet e ImageNet
2014
GAN y atención para traducción
2015–2016
ResNet
2017
Transformer
2018–2020
BERT y GPT-3 Preentrenamiento y escala
2020
Modelos de difusión modernos
Hito técnico
Contexto histórico de la IA
Orden cronológico con espaciado uniforme



Conexionismo y perceptrón

- El **conexionismo** estudia cómo una red de unidades simples puede producir un comportamiento complejo mediante sus conexiones.
- **En 1958**, Rosenblatt presentó un modelo probabilístico inspirado en percepción y aprendizaje.

Cómo decide un perceptrón

- Supón que x_1 y x_2 son dos mediciones de una pieza. La unidad calcula una puntuación z y luego aplica un escalón. Por convención, predice 1 si $z = 0$.

  $$z = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + b, \qquad \hat{y} = \mathbb{1}[z \geq 0].$$

- w_1, w_2 (pesos) determinan cómo influyen las entradas.
- b (bias) desplaza el umbral efectivo.
- **Calcula.** Con $\mathbf{x} = (1,0)$, $\mathbf{w} = (0.4,0.2)$ y $b = -0.5$, ¿qué clase predice?
  - La puntuación es $z = -0.1$ y la clase es 0.

Cómo corrige un error

- $e = y - \hat{y}, \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}, \qquad b \leftarrow b + \eta e.$
- Para un problema binario, un acierto da $e = 0$. Un falso negativo da $e = 1$ y un falso positivo da $e = -1$.
- **Inicio:** Con $\mathbf{x} = (1,0)$, $\mathbf{w} = (0.4,0.2)$ y $b = -0.5$.
- **Corrige** el falso negativo de la pieza anterior con $\eta = 0.1$.
  - $\mathbf{w}' = (0.5,0.2)$ y $b' = -0.4$ y la nueva puntuación es 0.1.
- La actualización puede cambiar las predicciones de otros ejemplos, ya que todos comparten pesos y sesgo.
- Con datos linealmente separables, la regla clásica converge a una separación después de un número finito de errores.

El primer invierno de la IA

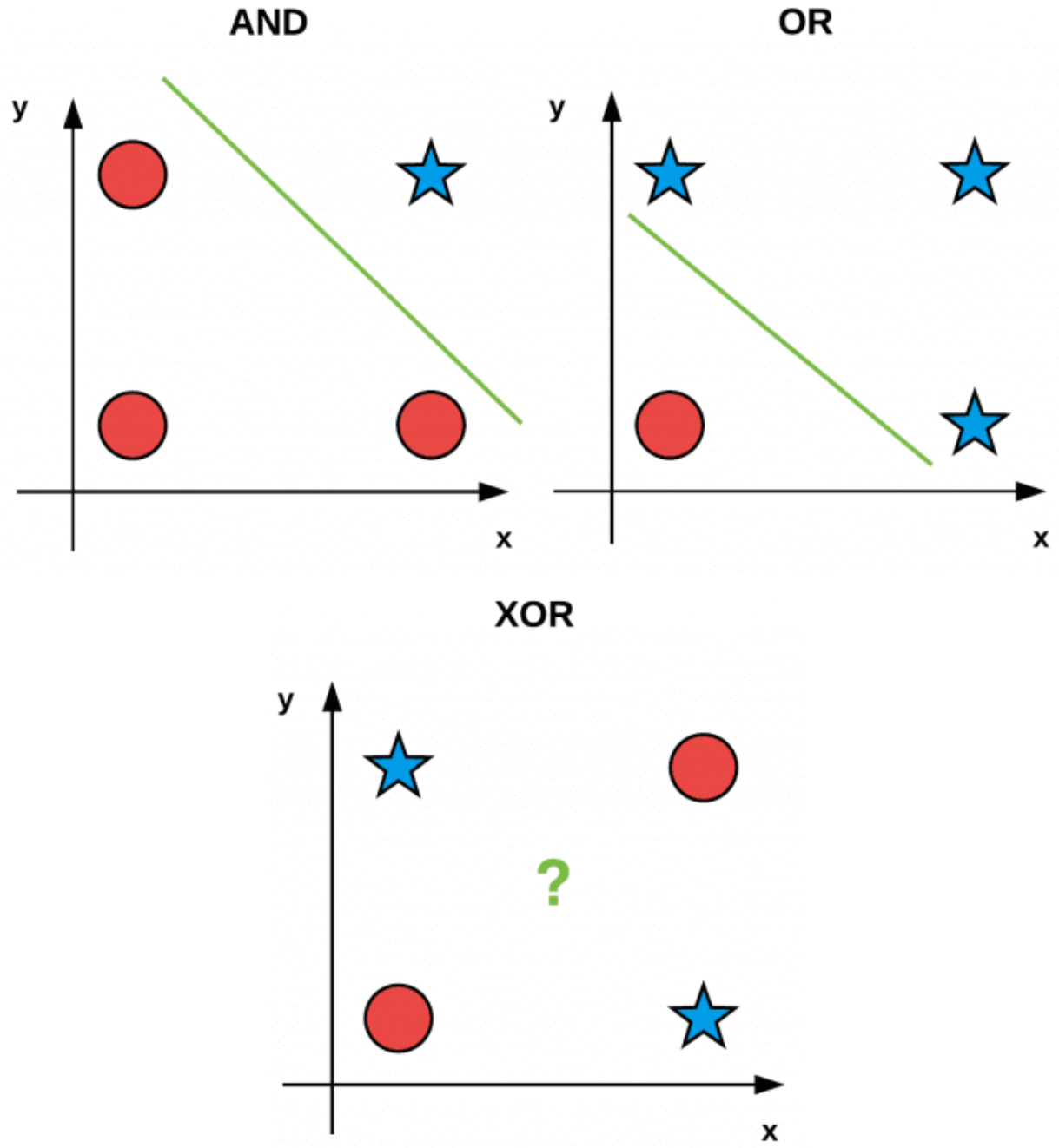
1969. Minsky y Papert analizan límites de ciertas arquitecturas de perceptrón (XOR).

1973. El informe Lighthill critica áreas de la investigación británica en IA.

Década de 1970. Crece el escepticismo ante promesas incumplidas y dificultades de escala.

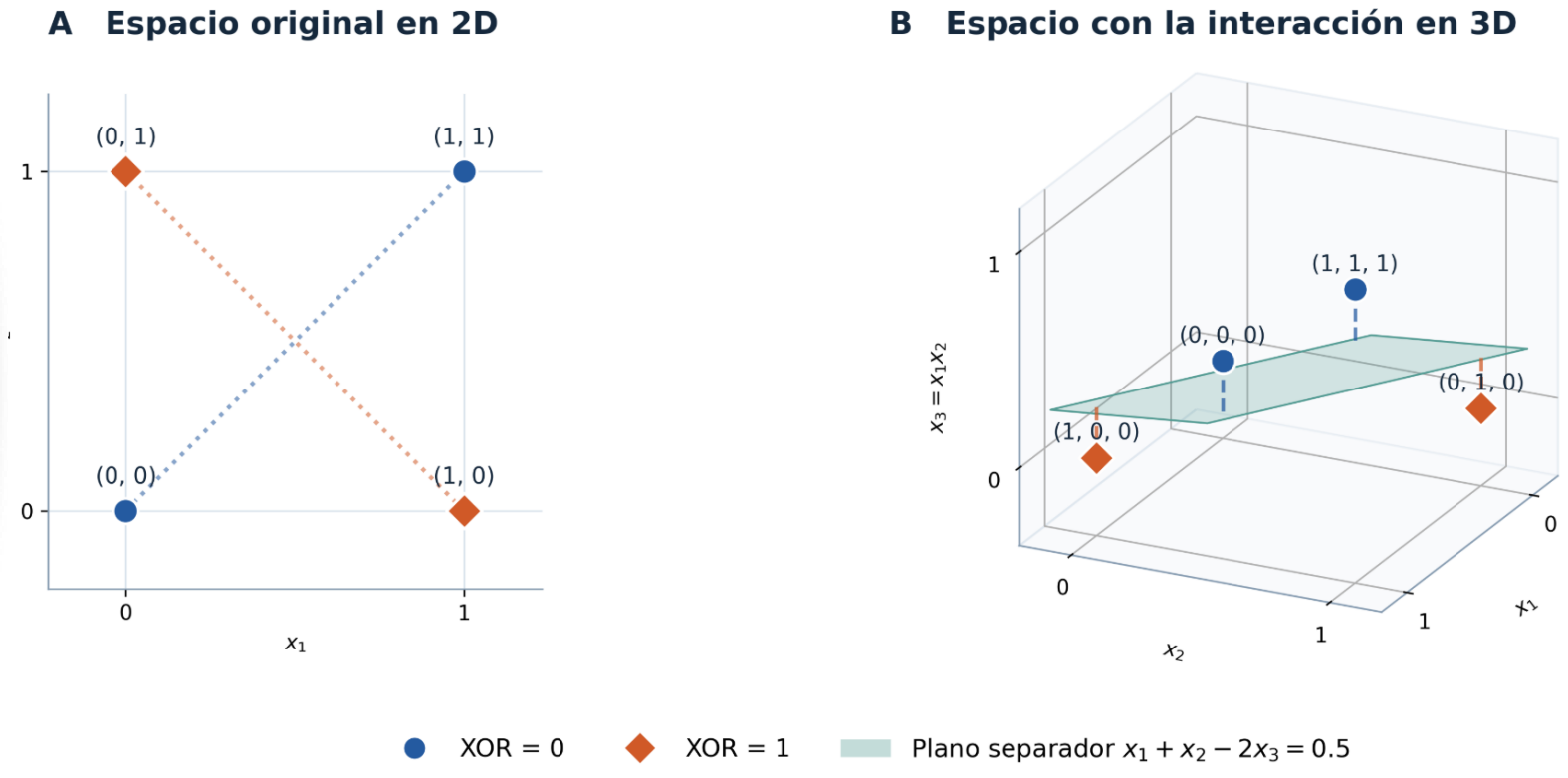
Las fronteras de AND, OR y XOR

- Un perceptrón puede separar los datos linealmente con una recta.
- ¿Pero que pasa para XOR?
- Las dos clases pasan a ocupar las diagonales. Sus segmentos se cruzan en el centro, por lo que una recta no puede separarlas.



Una característica cambia el problema

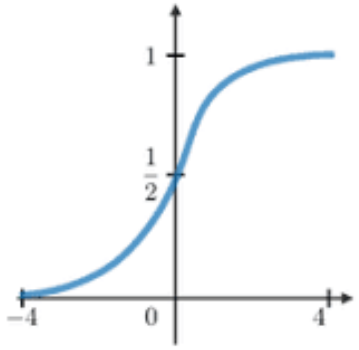
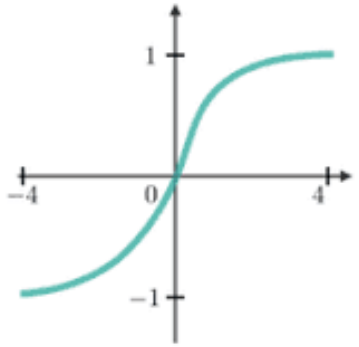
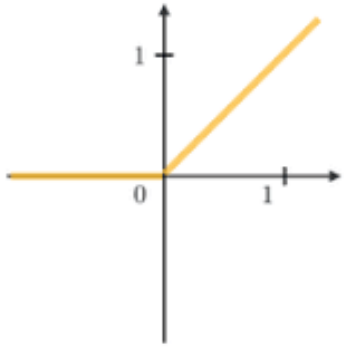
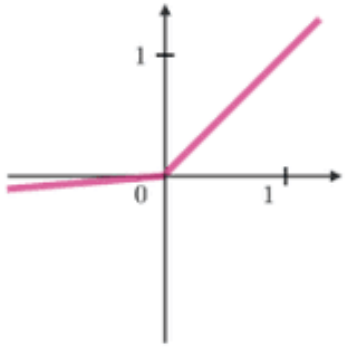
- **Añadiendo la interacción $x_1 x_2$.**
- Solo vale 1 cuando las dos entradas valen 1.
- La decisión vuelve a ser lineal en las variables ampliadas $(x_1, x_2, x_1 x_2)$.



Funciones de activación

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad a = g(z).$$

- Cada unidad calcula una combinación afín z y aplica una función de activación $a = g(z)$. La activación determina tanto la transformación que puede representar la unidad.

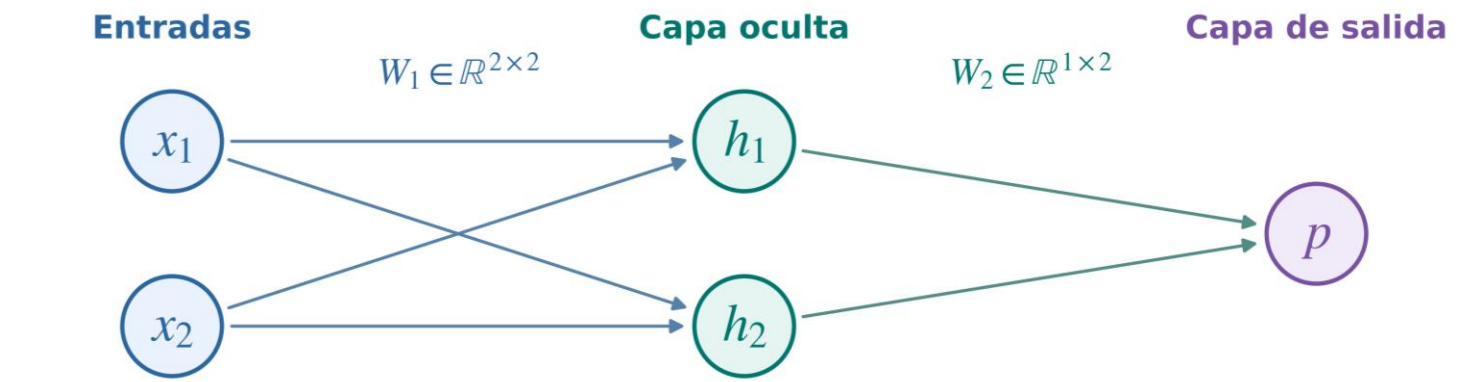
Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

Por qué importa la no linealidad

- Si todas las capas ocultas usan identidad, al componerlas obtenemos otra transformación afín. Para dos capas se puede ver agrupando términos.
  $W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$.
- Incluso si aplicamos una sigmoide a esa puntuación, con un umbral de 0.5 la frontera sigue siendo lineal. Una activación oculta no lineal permite transformar de manera distinta diferentes regiones de las entradas.

Multi Layer Perceptron (MLP)

- Un **perceptrón multicapa** o MLP encadena capas densas hacia delante.
- Cada unidad de una capa densa recibe todas las activaciones de la anterior.
- ¿Cuántos parámetros tiene esta red neuronal?



$$\mathbf{x} = [x_1, x_2]^T$$

Vector columna de entrada

$$\mathbf{h} = g(W_1\mathbf{x} + \mathbf{b}_1)$$

Activación de las dos unidades

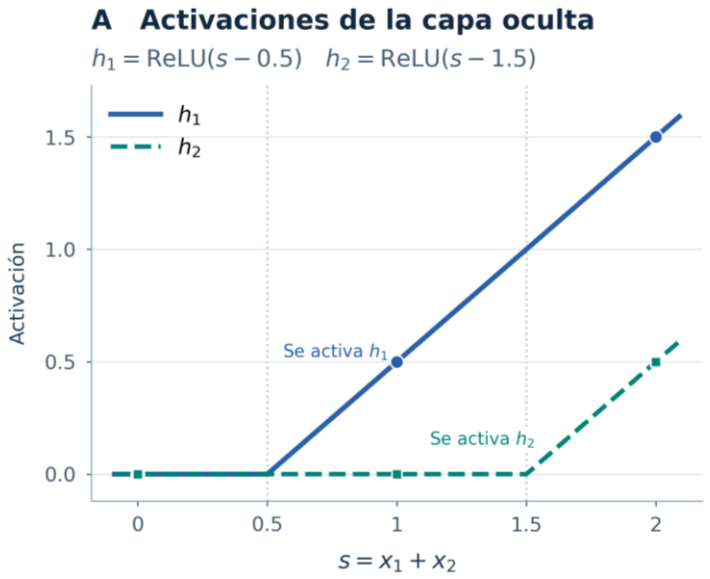
$$z_o = W_2\mathbf{h} + b_2$$
$$p = \sigma(z_o)$$

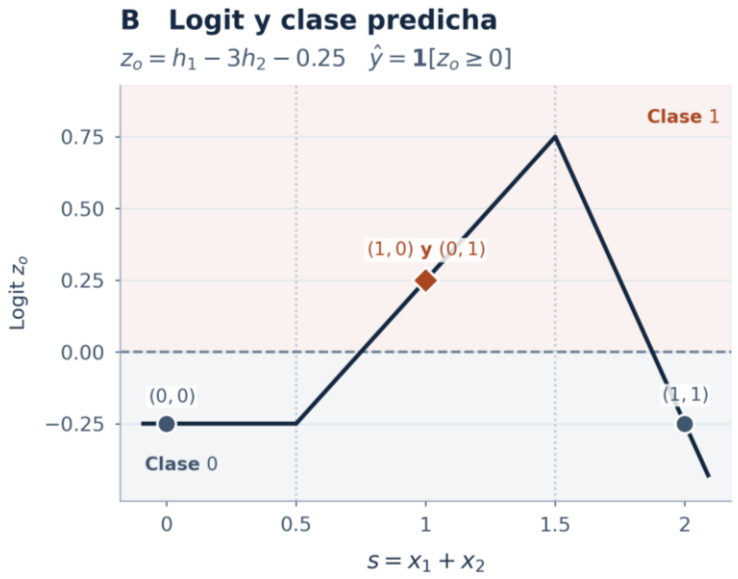
Logit y probabilidad estimada

Una MLP que resuelve XOR

$$s = x_1 + x_2, \qquad h_1 = \mathrm{ReLU}(s - 0.5)$$
$$h_2 = \mathrm{ReLU}(s - 1.5).$$
$$z_o = h_1 - 3h_2 - 0.25, \qquad \hat{y} = \mathbb{1}[z_o \geq 0].$$

Entrada	h_1	h_2	z_o	Clase
$(0,0)$	0	0	−0.25	0
$(1,0)$	0.5	0	0.25	1
$(0,1)$	0.5	0	0.25	1
$(1,1)$	1.5	0.5	−0.25	0





La pérdida distingue entre aciertos

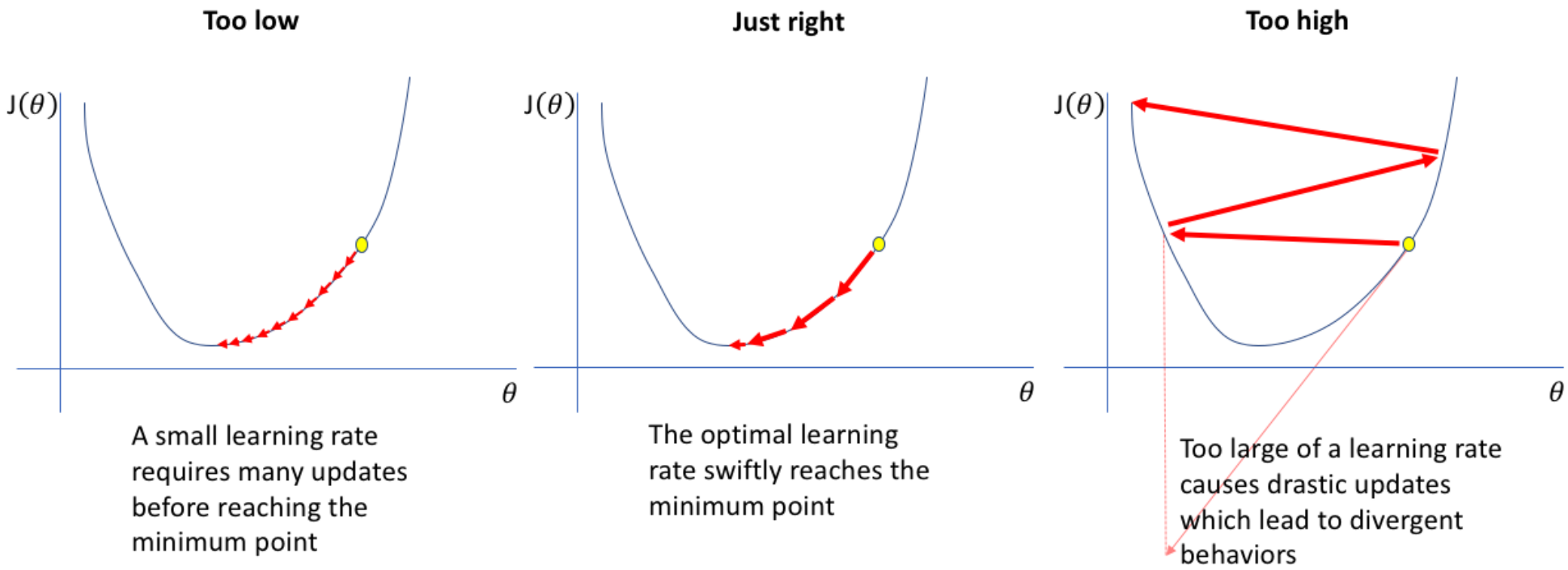
- La **entropía cruzada binaria** penaliza una probabilidad baja para la etiqueta observada.

$$\ell(y,p) = -[y\log p + (1-y)\log(1-p)], \qquad J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\,(y_i, p_i).$$

- Para $y = 1$, las predicciones $p = 0.6$ y $p = 0.9$ dan la misma clase con umbral 0.5, pero sus pérdidas son aproximadamente 0.511 y 0.105.
- La exactitud cuenta aciertos después de aplicar un umbral. La pérdida distingue predicciones con grados de confianza distintos.

Descenso del gradiente

- Es un método que **ajusta los pesos de la red paso a paso para reducir la pérdida**.
- El **gradiente** indica hacia dónde aumenta más rápido la pérdida.
- Movemos los pesos en la **dirección contraria**.
- La **tasa de aprendizaje η** controla el tamaño de cada paso.



Ejemplo del descenso del gradiente

- Una neurona lineal recibe $x_1 = 1$ y $x_2 = 2$. Sus pesos iniciales son $w_1 = w_2 = 0.5$ y el valor objetivo es $y = 2$. Haremos una actualización con tasa de aprendizaje $\eta = 0.1$.
- **1. Calcular la predicción y la pérdida**
  - Multiplicamos cada entrada por su peso y sumamos los resultados.
  - $\hat{y} = w_1 x_1 + w_2 x_2 = 0.5(1) + 0.5(2) = 1.5.$
  - $L = 1/2\,(\hat{y} - y)^2 = 1/2\,(1.5 - 2)^2 = 0.125.$
- **2. Calcular el gradiente de cada peso**
  - La diferencia entre la predicción y el objetivo es -0.5. Para esta pérdida y esta neurona, multiplicamos esa diferencia por la entrada de cada peso.
  - $\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1 = (-0.5)(1) = -0.5, \qquad \frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2 = (-0.5)(2) = -1.$

Ejemplo del descenso del gradiente

- **3. Actualizar los pesos**
  - Restamos a cada peso su gradiente multiplicado por la tasa de aprendizaje.
  - $w_i^{\text{nuevo}} = w_i - \eta \frac{\partial L}{\partial w_i}$.

Peso	Antes	Actualización	Después
w_1	0.50	$0.50 - 0.1(-0.5)$	0.55
w_2	0.50	$0.50 - 0.1(-1)$	0.60

- Con los pesos nuevos, $\hat{y} = 0.55(1) + 0.60(2) = 1.75$ y $L = 1/2\,(1.75 - 2)^2 = 0.03125$. La pérdida bajó de **0.125 a 0.03125**.

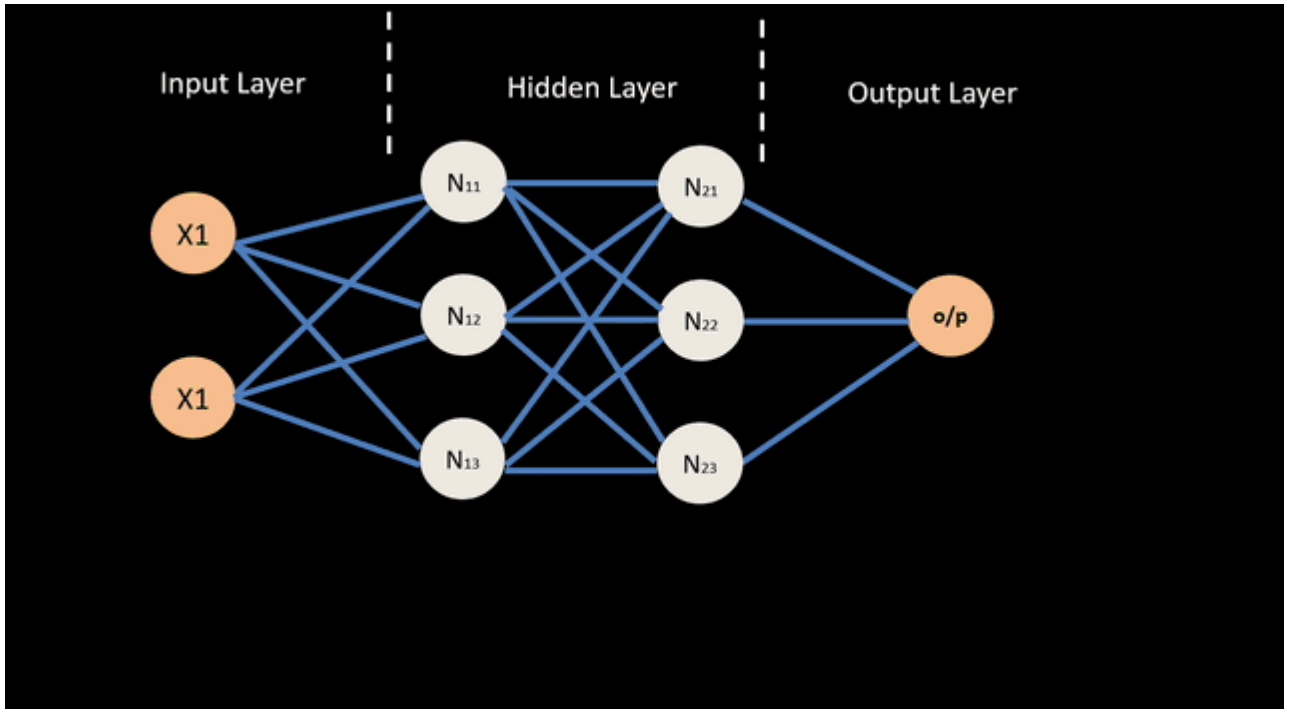
Lotes y épocas

- Una **época** es un recorrido por los datos de entrenamiento. Una **iteración** suele referirse a una actualización.

Método	Datos por paso	Pasos por época con 100 observaciones
Lote completo	Todo el conjunto	1
Estocástico estricto	Una observación	100
Batch	Un subconjunto	5 con lotes de 20

El resurgimiento conexionista

- En los años ochenta crecen los sistemas expertos basados en reglas.
- **En 1986**, Rumelhart, Hinton y Williams difunden el entrenamiento de capas ocultas con backpropagation reduciendo el costo de actualización de los pesos de $O(n^2)$ a $\mathrm{O(n)}$.



Backpropagation

- **Pasada hacia delante.** Calcula preactivaciones, activaciones, predicción y pérdida.
- **Backpropagation.** Aplica la regla de la cadena desde la pérdida hacia las entradas. Multiplica sensibilidades a lo largo de cada ruta y suma contribuciones si un parámetro influye por varias rutas.
- **Actualización.** El optimizador modifica los parámetros. Los gradientes de un mismo paso se calculan con los pesos anteriores.

El segundo invierno de la IA

A finales de los ochenta y principios de los noventa se enfrió parte del mercado de IA, sobre todo alrededor de los sistemas expertos.

Mantener reglas y adaptarlas a casos nuevos podía ser costoso.

En esos años siguieron desarrollándose técnicas de aprendizaje y aplicaciones de reconocimiento.

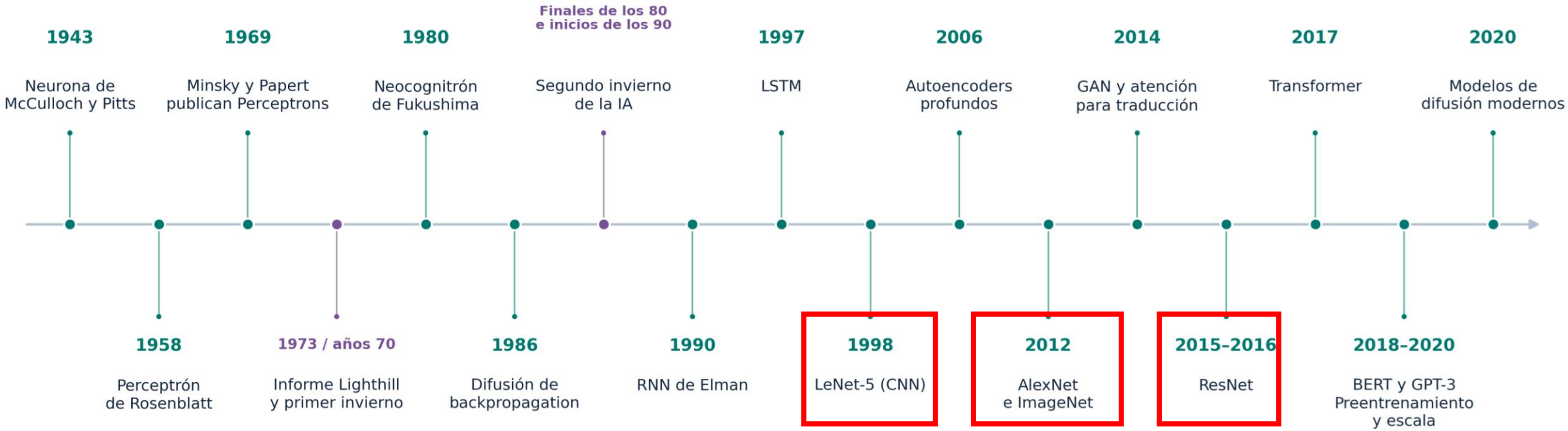
¿Qué separa un prototipo exitoso de un sistema útil y mantenible?

Evaluación y diagnóstico

- **Reserva el conjunto de prueba desde el inicio.** Usarlo al final, cuando se haya fijado la configuración del modelo.
- **Elegir la configuración con validación.** La validación cruzada requiere entrenar una vez por pliegue para cada configuración. Si ese costo es alto, usa una **partición fija en entrenamiento, validación y prueba**.
- **Distinguir entre parámetros e hiperparámetros.** La red aprende los pesos y sesgos. La arquitectura, la activación, la tasa de aprendizaje, el tamaño del lote y la regularización se eligen con validación.
- **Prepara las entradas.** Si escalas las entradas, ajusta el escalador solo con los datos de entrenamiento de la partición o del pliegue correspondiente.

MLP en imágenes

- **MLP aprende características** al combinar las entradas en sus capas ocultas.
- **En una imagen, la posición importa.** Si aplanamos la imagen para alimentar una capa densa, cada posición de píxel tiene su propio peso hacia cada neurona.
- Un mismo patrón puede aparecer en distintos lugares.
- ¿Podemos aprender un solo detector de bordes y reutilizarlo en toda la imagen?
- Esa es la idea que exploraremos con las **redes neuronales convolucionales (CNN)**.


1943
Neurona de McCulloch y Pitts
1958
Perceptrón de Rosenblatt
1969
Minsky y Papert publican Perceptrons
1973 / años 70
Informe Lighthill y primer invierno
1980
Neocognitrón de Fukushima
1986
Difusión de backpropagation
Finales de los 80 e inicios de los 90
Segundo invierno de la IA
1990
RNN de Elman
1997
LSTM
1998
LeNet-5 (CNN)
2006
Autoencoders profundos
2012
AlexNet e ImageNet
2014
GAN y atención para traducción
2015–2016
ResNet
2017
Transformer
2018–2020
BERT y GPT-3 Preentrenamiento y escala
2020
Modelos de difusión modernos
Hito técnico
Contexto histórico de la IA
Orden cronológico con espaciado uniforme